

29/8/26 DAY 6:

Namespaces

- **Namespaces** is a declarative region that provides a scope to the identifiers (names of types, functions, variables, etc) inside it.
- Namespaces are used to organize code into logical groups and to prevent name collisions that can occur especially when your code base includes multiple libraries.
- **Syntax:**

```
namespace namespace_name {  
    // code declarations  
}
```

- It can be accessed using the scope resolution operator `::` or by using the `using` directive.
- It can consist of nested namespaces, and can be defined in multiple files.
- The namespace can have variables, functions, classes, and other namespaces.

Nested Namespaces

- Nested namespaces are namespaces defined within another namespace.
- They allow for better organization and encapsulation of code, especially in large projects.
- Syntax:

```
// old way  
namespace outer_namespace {  
    namespace inner_namespace {  
        // code declarations  
    }  
}  
  
// modern way (C++17 and later)  
namespace outer_namespace::inner_namespace {  
    // code declarations  
}
```

std Namespace

The **std** namespace is a built-in namespace in C++ that contains all the standard library components.

- It is widely spread in different parts of the C++ standard library, including headers like `<iostream>`, `<vector>`, `<string>`, `<map>`, and many others.
- It includes classes, functions, and objects for input/output operations, data structures, algorithms, and more.
- Commonly used components in the std namespace include:

- `std::cout` for output
- `std::cin` for input
- `std::vector` for dynamic arrays
- `std::string` for strings
- `std::map` for associative arrays
- The **std** namespace is automatically included when you include standard library headers, but you can also use the **using** directive to avoid prefixing with **std::**.

The `std` namespace is essential for writing modern C++ code, as it provides a wide range of tools and utilities that simplify programming tasks and enhance code readability and maintainability.

Different Scopes

Scope refers to the region of a program where a defined variable can be accessed. In C++, there are several types of scopes:

1. **Global Scope:** Variables declared outside of all functions and classes have global scope.
2. **Local Scope:** Variables declared within a function or block have local scope and are only accessible within that function or block.
3. **Class Scope:** Variables declared within a class are accessible to all member functions of that class.
4. **Namespace Scope:** Variables declared within a namespace are accessible to all functions and classes within that namespace.
5. **Block Scope:** Variables declared within a block (enclosed by `{}`) are only accessible within that block.
6. **Function Scope:** Variables declared within a function are only accessible within that function.

Header files

Header files in C++ are used to include the declarations of functions, classes, and variables that are defined in other files. They are typically used to share code between multiple source files.

- Header files usually have a `.h` or `.hpp` extension and are included in source files using the `#include` preprocessor directive.

`#include` is include directive that tells the preprocessor to include the contents of a specified file into the current file before compilation. It is used to include header files, which contain declarations of functions, classes, and variables that are defined in other files.

- When the project is complex and has multiple source files, header files help in organizing the code and avoiding redundancy. They allow for better code management and modularity.

Syntax:

```
#include <header_file> // for standard library headers
#include "header_file" // for user-defined headers
```

Header files with folder structure

When working with a project that has a folder structure, you can include header files from different directories by specifying the relative or absolute path to the header file.

Example:

```
#include "folder_name/header_file.h" // relative path
#include "/absolute/path/to/header_file.h" // absolute path
```

Absolute paths are less common and can make the code less portable, so relative paths are generally preferred.

Header guards

Header guards are preprocessor directives that prevent the same header file from being included multiple times in a single compilation unit. They are used to avoid redefinition errors.

- When the preprocessor encounters a header file, it checks if a unique identifier (macro) has been defined.
- If it has not been defined, the preprocessor defines it and includes the contents of the header file.
- If it has already been defined, the preprocessor skips the contents of the header file.

```
#ifndef HEADER_FILE_H
#define HEADER_FILE_H

// Header file content

#endif // HEADER_FILE_H
```

Modern C++ (C++11 and later) also supports `#pragma once` as an alternative to header guards. It is a non-standard but widely supported preprocessor directive that serves the same purpose of preventing multiple inclusions of a header file.

```
#pragma once
// Header file content
```

#include < > vs " "

The difference between `#include <header_file>` and `#include "header_file"` is that the former is used for standard library headers, while the latter is used for user-defined headers.

We can also use `#include < >` for user-defined headers and `#include " "` for standard library headers, but it is not recommended as it may lead to confusion and potential conflicts with standard library headers.

Dynamic 1-D arrays

```
#include <iostream>
int main() {
    int size;
    std::cout << "Enter the size of the array: ";
    std::cin >> size;

    // Dynamically allocate memory for a 1-D array
    int* arr = new int[size];

    // Input values into the array
    std::cout << "Enter " << size << " elements:\n";
    for (int i = 0; i < size; ++i) {
        std::cin >> arr[i];
    }

    // Output the values in the array
    std::cout << "You entered:\n";
    for (int i = 0; i < size; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    // Deallocate memory
    delete[] arr;

    return 0;
}
```

Dynamic 2-D arrays

```
#include <iostream>
using namespace std;
int main() {
    int rows, cols;
    cout << "Enter the number of rows: ";
    cin >> rows;
    cout << "Enter the number of columns: ";
    cin >> cols;

    // Dynamically allocate memory for a 2-D array
    int** arr = new int*[rows];
    for (int i = 0; i < rows; ++i) {
        arr[i] = new int[cols];
    }

    // Input values into the array
    cout << "Enter elements of the array:\n";
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cin >> arr[i][j];
        }
    }
}
```

```

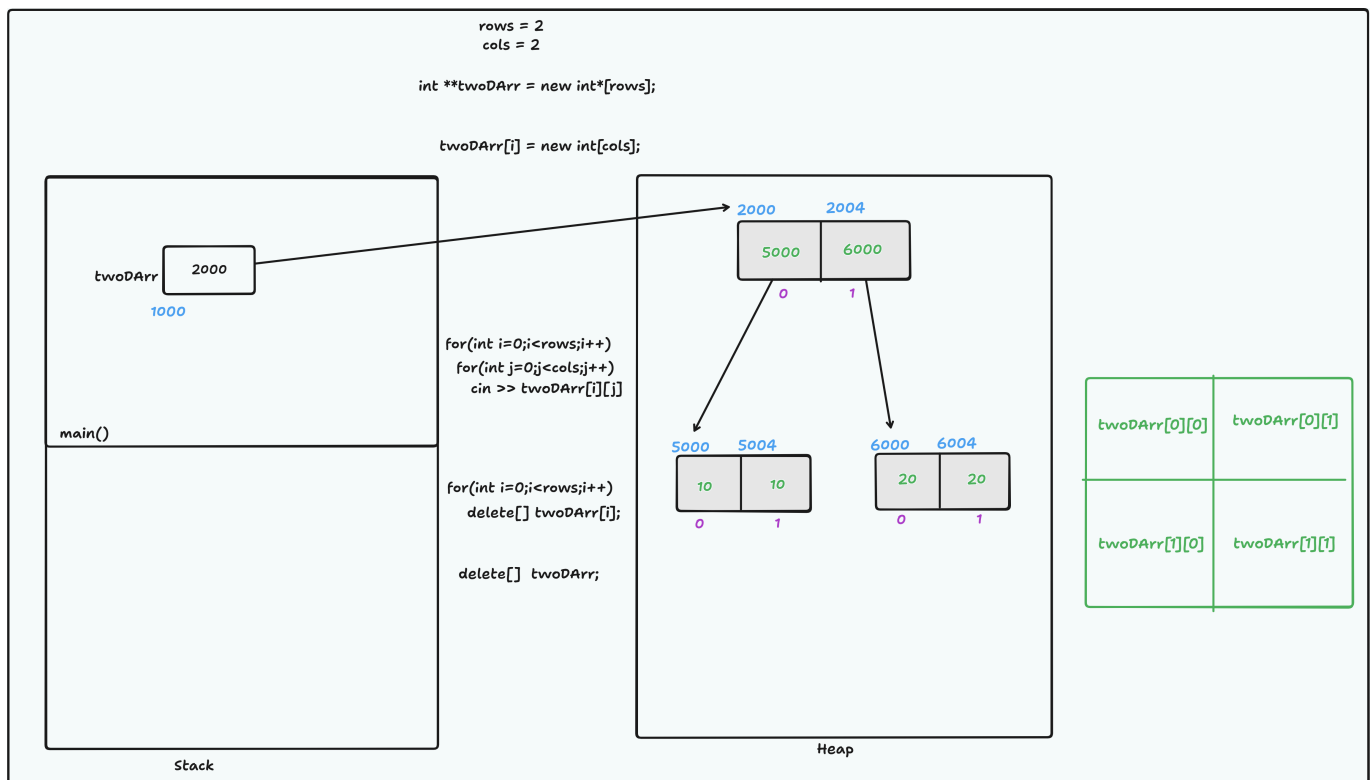
    }
}

// Output the values in the array
cout << "You entered:\n";
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; ++j) {
        cout << arr[i][j] << " ";
    }
    cout << endl;
}

// Deallocate memory
for (int i = 0; i < rows; ++i) {
    delete[] arr[i];
}
delete[] arr;

return 0;
}

```



Pointer to Objects (Class Pointer)

It is a pointer that points to an object of a class. It is used to access the members of the class using the arrow operator ($\rightarrow$).

It is used when we create an object of a class dynamically using the **new** operator.

```

class Student {
public:

```

```

    int rollNo;
    void printdetails() {
        std::cout << "Roll No: " << rollNo << std::endl;
    }
};

int main() {
    Student obj;
    obj.rollNo = 42;

    // Create a pointer to the object
    Student* ptr = &obj;

    // Access members using the arrow operator
    ptr->printdetails();

    // Create a dynamic object of the class: is a pointer to the object
    Student* dynamicObj = new Student();
    // access members using the arrow operator
    dynamicObj->printdetails();
    dynamicObj->rollNo = 100;

    return 0;
}

```

this Pointer

- It points to the **current object** of the class.
- It is constant pointer, meaning it cannot be reassigned to point to another object.
- It is a special pointer that is available to all **non-static member functions** of a class.
- It points to the object for which the member function is called.
- It is used to access the members of the object and to resolve naming conflicts between member variables and parameters.
- It is also used to return the current object from a member function, which allows for **method chaining**.
- It is used to pass the current object as an argument to another function.

```

#include <iostream>
using namespace std;

class Student{
    int rollno;
    string name;
    int age;
public:

    Student(){

    }

    Student(int rollno, string name, int age){

```

```

        this->rollno = rollno;
        this->name = name;
        this->age = age;
    }

    int getAge()const{
        return age;
    }

// int getAge(Student const *this) const {
//     age = 10;    // not allowed
//     return age;
// }

// void setAge(Student*const this, int age) {
//     this->age = age;
// }

    const string& getName() const {
        return name;
    }
    int getRollno() const {
        return rollno;
    }

    Student& setName(const string &name) {
        this->name = name;
        return *this;
    }

    Student& setAge(int age) {
//     cout << *this << endl;
        this->age = age;
        return *this;
    }

    void setRollno(int rollno) {
        this->rollno = rollno;
    }
};

int main() {
    Student s;
    Student s1;

//     cout << s1.getRollno();
//     s1 = address = *this

//     s1.getName();
//     Student* getName(&s1);

//     Student *const this;        // & cannot change

```

```
s1.setName("shil").setAge(30);

cout << s1.getName() << " " << s1.getAge() << endl;

return 0;
}
```

Once the object is created, the **this** pointer is automatically associated with it that cannot be reassigned.

Tomorrow's Topics:

- Enumeration
- typedef
- Type Aliases
- using type aliases
- Resource Ownership Concept
- Memory Leak
- Dangling Pointer
- Double Deletion
- RAII Concept
- Introduction to Smart Pointers
- Why Smart Pointers?